Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки

ЗВІТ

з лабораторної роботи № 5
з дисципліни «Основи програмування»
«Умовні оператори»

Виконала:
студентка групи ФЕП-13
Савчак Дарина
Перевірив:
асистент Васюта В. М.

Львів 2026

Мета роботи:

Вивчити поняття і застосування умовних операторів.

Обладнання та програмне забезпечення:

– IBM сумісна персональна обчислювальна машина;

– онлайн компілятор мови програмування Сі, доступний за посиланням

https://repl.it/languages/c

Завдання до роботи:

Написати програму на мові програмування Сі, котра реалізує наступні функції:

– використання оператора if;

– використання оператора if – else – if;

– використання оператора ?;

– використання оператора switch;

– використання вкладених операторів switch;

Теоретичні відомості:

1. **Оператор if**
   Оператор if використовується для реалізації розгалужень у програмі та має таку структуру:
   if (умова) оператор;
   else оператор;
   Для виконання кількох операторів використовується розширена форма запису у фігурних дужках:
   if (умова) {
   блок операторів;
   }
   else {
   блок операторів;
   }
2. **Вкладені оператори if**
   Це конструкції, де умовний оператор розташовується всередині іншого оператора if або else:

```
if (умова1) {
    if (умова2) оператор;
}
else {
     блок операторів;
}
```

3. **Конструкція if-else-if**

   Використовується для послідовної перевірки кількох взаємовиключних умов:

```
if (умова1) {
   блок операторів;
}
else if (умова2) {
     блок операторів;
}
 else if (умова3) {
      блок операторів;
}
else {
     блок операторів;
}
```

4. **Тернарний оператор ?**

   Скорочена форма умовного оператора, яка повертає значення залежно від умови:

```
змінна = умова ? вираз1 : вираз2;
// Приклад:
y = x > 1 ? 10 : 20;
```

   Також може використовуватися для виконання різних команд:

```
умова ? оператор1 : оператор2;
```

5. **Оператор switch**

   Використовується для організації множинного вибору залежно від значення виразу:

```
switch (вираз) {
     case константа1:
        послідовність операторів;
        break;
  case константа2:
```

послідовність операторів;
break;
default:
послідовність операторів;
}

6. **Вкладені оператори switch**

У мові Сі допускається використання вкладених конструкцій switch, коли один оператор перемикача розміщується всередині гілки case іншого оператора switch.

ХІД РОБОТИ

2. **Написати програму, котра видаватиме результат порівняння двох введених з клавіатури чисел з допомогою оператора «?».**

Для написання програми я спершу підключила стандартну заголовочну бібліотеку вводу-виводу <stdio.h>, яка необхідна для виведення тексту на екран та зчитування даних. У головній функції int main(void) я оголосила дві цілочислові змінні a та b, після чого за допомогою функції printf вивела запрошення до введення, а за допомогою scanf зчитала значення, введені з клавіатури.

Оскільки головною вимогою завдання було застосування саме тернарного оператора, я використала вкладену конструкцію для перевірки всіх можливих випадків взаємозв'язку між числами. Текст самої програми виглядає так:

```
#include <stdio.h>

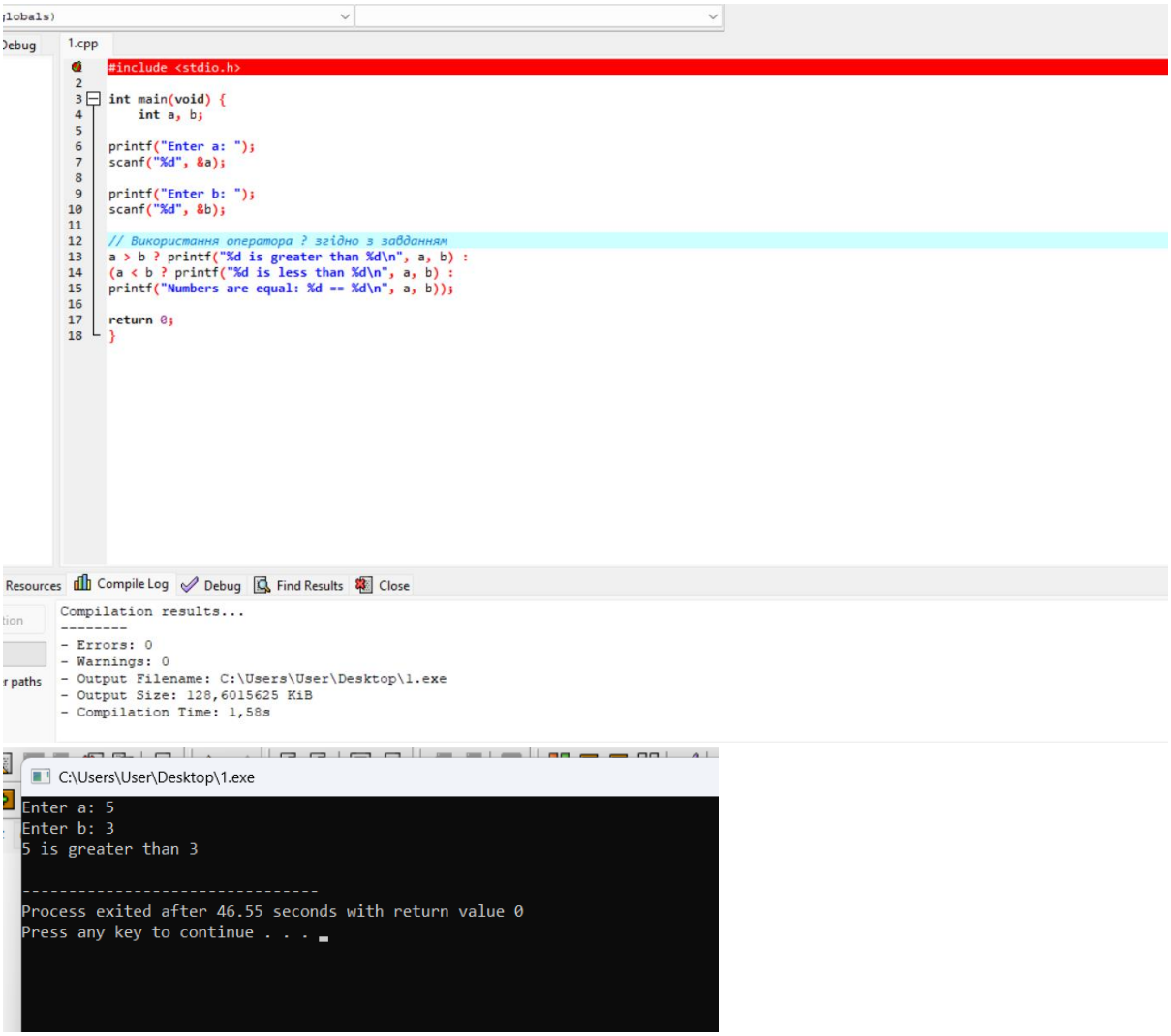
int main(void) {
    int a, b;
    printf("Enter a: ");
    scanf("%d", &a);

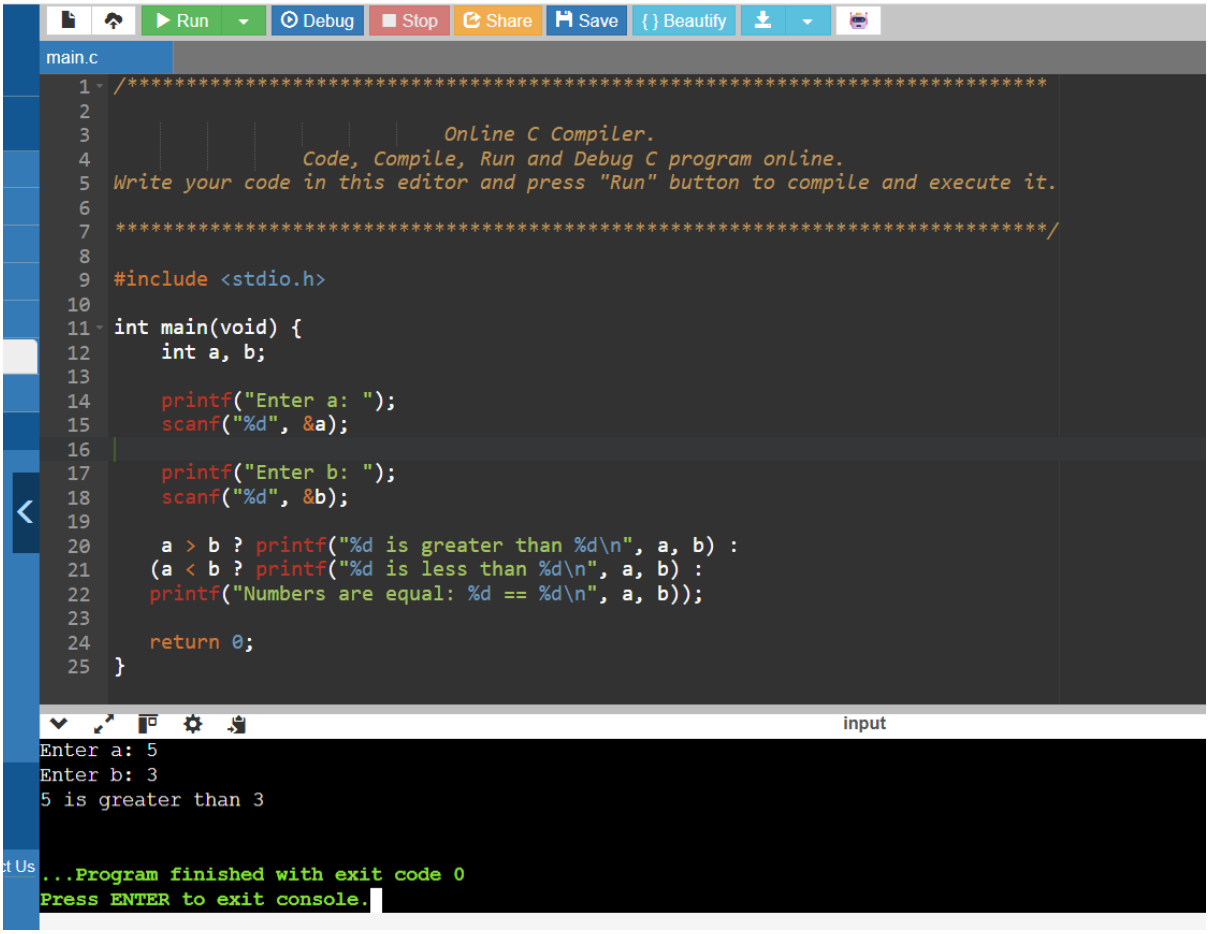
    printf("Enter b: ");
    scanf("%d", &b);

a > b ? printf("%d is greater than %d\n", a, b) :
(a < b ? printf("%d is less than %d\n", a, b) :
printf("Numbers are equal: %d == %d\n", a, b));
```

return 0;
}

Працює цей код наступним чином: спочатку програма перевіряє умову a > b. Якщо вона виконується, спрацьовує перший вираз і виводиться повідомлення про те, що перше число більше за друге. Якщо ця умова виявляється хидною, програма переходить до другої частини через двокрапку, де перевіряється наступна умова — a < b. Якщо і вона хибна, це означає, що числа рівні, тому спрацьовує фінальна гілка з повідомленням про рівність.
Після написання я успішно скомпільовала проєкт у середовищі Dev-C++, переконавшись відсутність помилок і попереджень (Errors: 0, Warnings: 0), протестувала роботу програми для різних комбінацій чисел та зберегла скріншоти консолі для оформлення звіту.





https://onlinegdb.com/fA8r1bowQ9

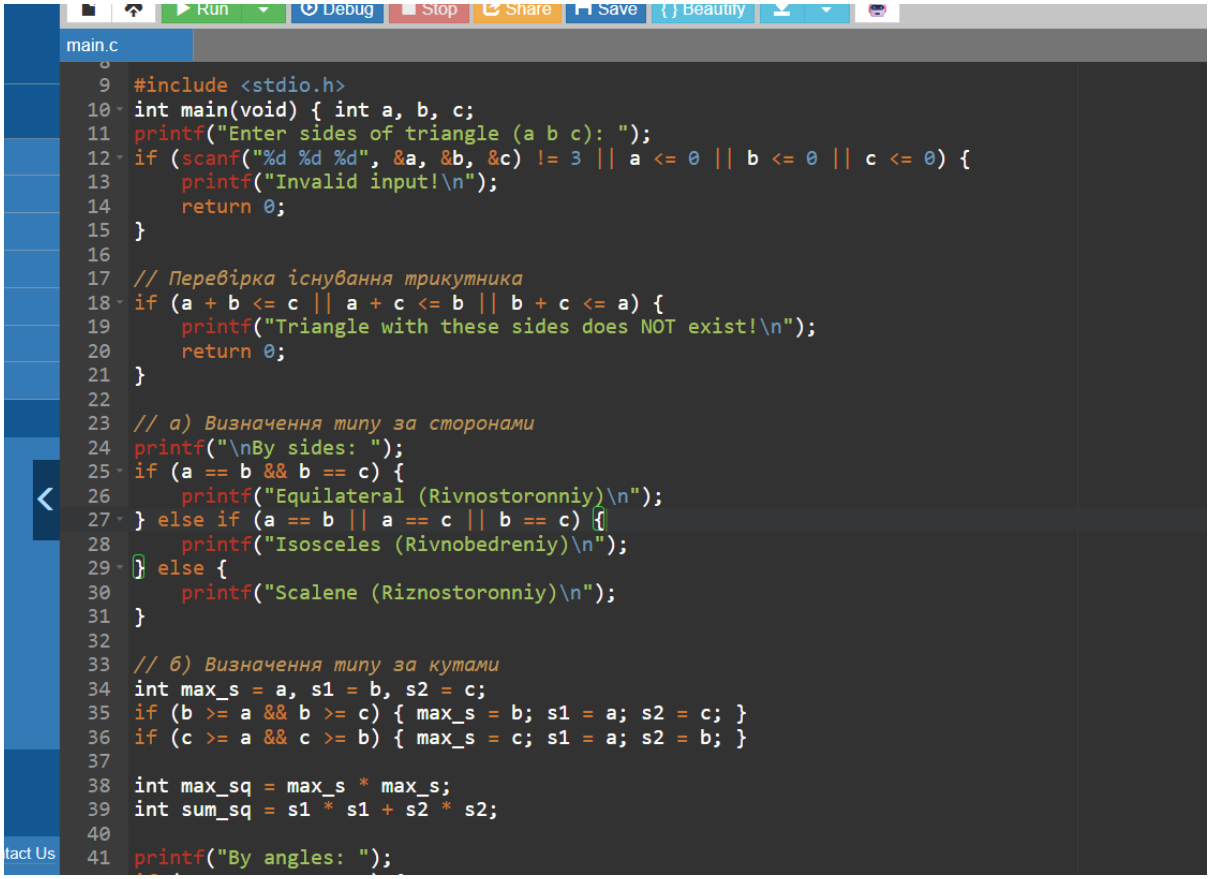
<script src="//onlinegdb.com/embed/js/sBGc5A-g8?theme=dark"></script>

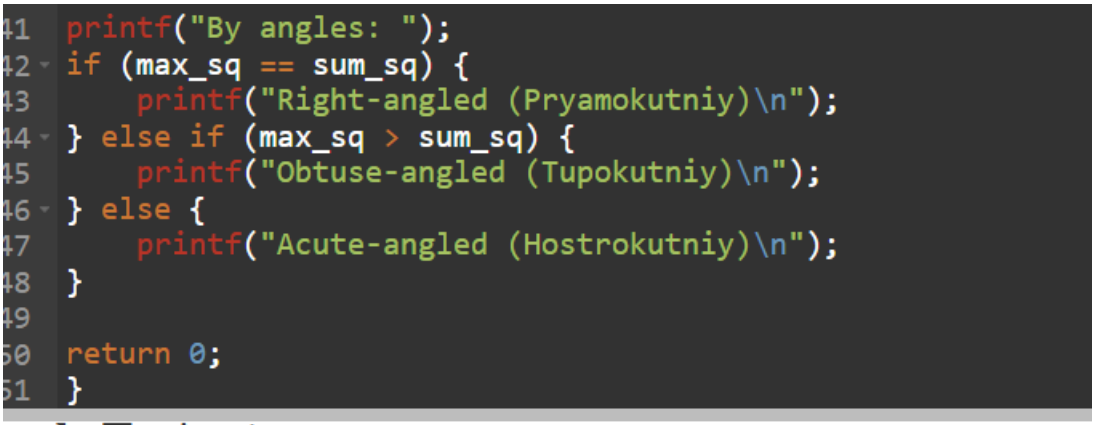
3. **Написати програму, котра дозволяє ввести з консолі довжини трьох сторін трикутника (наприклад, 3 4 5) і визначити його тип: а) рівносторонній (правильний), рівнобедренний, різносторонній; б) прямо-, гостро-, тупо-кутний.**

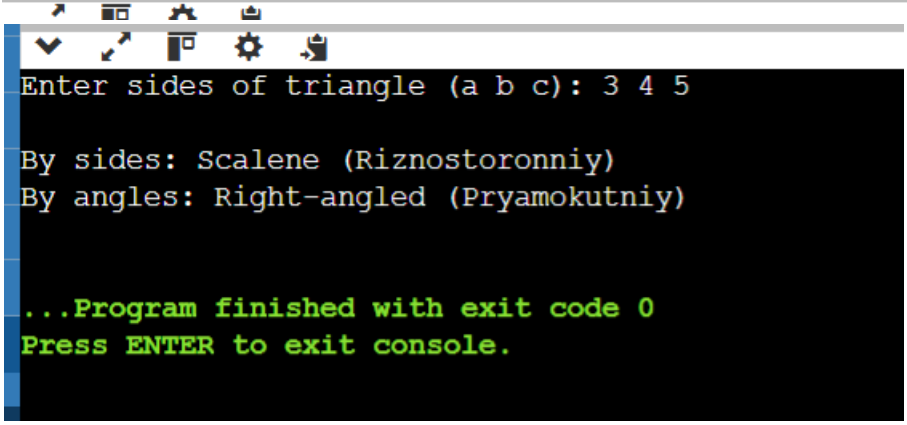
Для виконання Завдання № 3 я розробила програму мовою С, яка аналізує введені з клавіатури довжини трьох сторін трикутника, перевіряє, чи може такий трикутник існувати, та визначає його тип за сторонами і кутами. Роботу я виконувала в середовищі Dev-C++ та додатково перевірила код в онлайн-середовищі OnlineGDB.

У функції main я оголосила три цілочислові змінні a, b та c для збереження сторін. Далі за допомогою scanf програма зчитує введені числа. Я одразу додала перевірку: якщо користувач вводить некоректні дані або числа, менші чи рівні нулю, програма виводить повідомлення про помилку та завершує роботу.

Після цього програма перевіряє основне правило існування трикутника: сума двох будь-яких його сторін обов'язково має бути більшою за третю сторону **(a + b > c, a + c > b, b + c > a)**. Якщо хоча б одна умова не виконується, програма виводить повідомлення, що трикутник із такими сторонами не існує, і зупиняється.
Якщо ж трикутник існує, програма переходить до визначення його типу:
Визначення типу за сторонами: за допомогою стандартної конструкції if-else порівнюються між собою всі сторони. Якщо всі три сторони рівні **(a == b && b == c)**, програма виводить, що трикутник **рівносторонній (Equilateral)**. Якщо рівні лише дві сторони, то він **рівнобедрений (Isosceles),** а якщо всі сторони різні — **різносторонній (Scalene).**
Визначення типу за кутами: для цього я використала теорему, обернену до теореми Піфагора. Спочатку програма знаходить найбільшу сторону та обчислює її квадрат, а також рахує суму квадратів двох інших сторін. Далі порівнюються ці значення: якщо квадрат найбільшої сторони дорівнює сумі квадратів двох інших (max_sq == sum_sq), трикутник є прямокутним (Right-angled). Якщо квадрат найбільшої сторони більший за суму квадратів двох інших — тупокутним (Obtuse-angled), а якщо менший — гострокутним (Acute-angled).
Текст написаної програми виглядає наступним чином:







https://onlinegdb.com/GSA016zRA0

<script src="//onlinegdb.com/embed/js/GSA016zRA0?theme=dark"></script>

Я успішно скомпільовала та протестувала цю програму. Під час введення тестових даних 3 4 5 програма правильно визначила, що такий трикутник є різностороннім (Scalene) та прямокутним (Right-angled). Робота програми пройшла без помилок, результати збережено на скріншоті та загенеровано посилання на код.

4. **Написати програму, котра дозволяє ввести з консолі тризначне число і вивести його прописом.**

Для виконання Завдання № 4 я розробила програму мовою С, яка приймає з консолі тризначне ціле число у діапазоні від 100 до 999 та виводить його текстову назву прописом українською мовою.

У функції main я оголосила цілочислову змінну num для збереження введеного числа. За допомогою функції scanf програма зчитує значення з консолі. Для забезпечення надійності алгоритму я відразу реалізувала перевірку вхідних даних: якщо користувач вводить нечислові символи або число поза межами діапазону [100, 999] (наприклад, менше 100 чи більше 999), програма виводить повідомлення про помилку та завершує виконання.

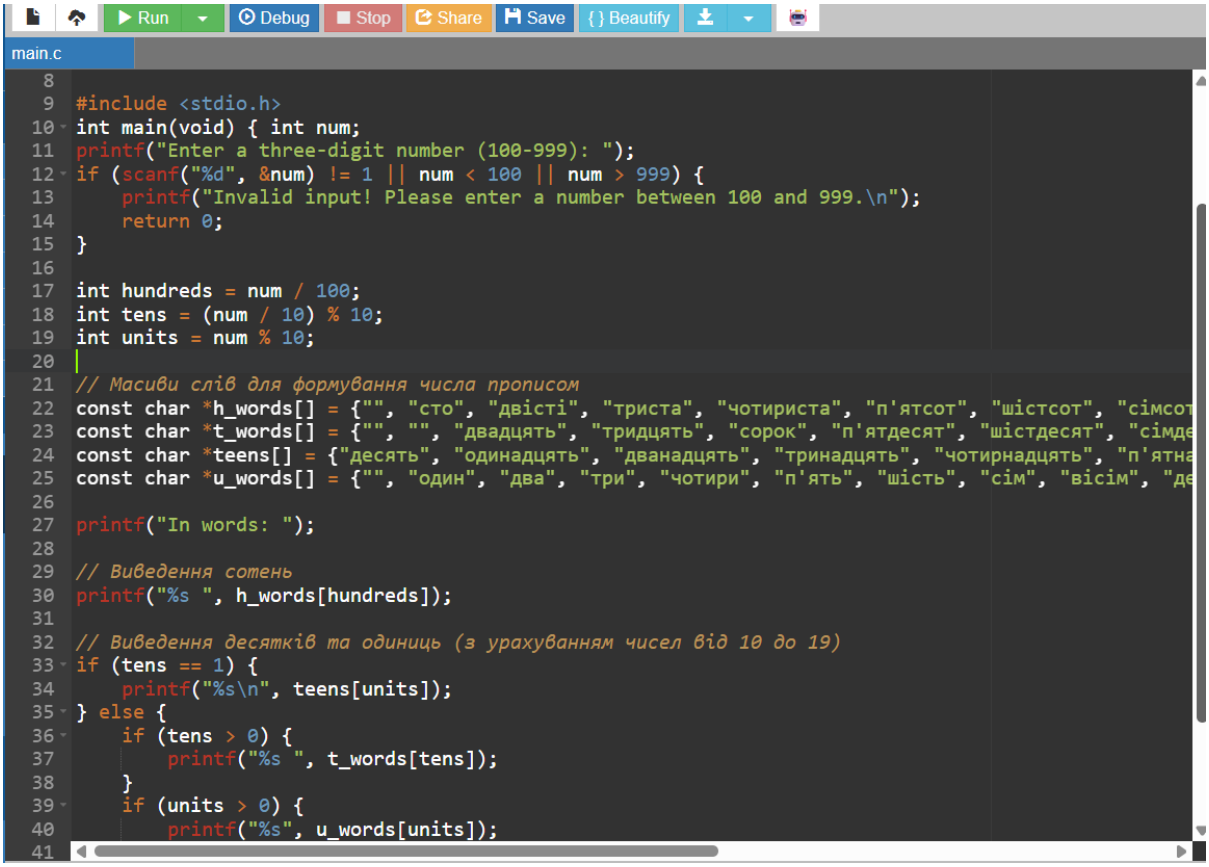
Після успішного введення числа програма розбиває його на три окремі розряди за допомогою математичних операцій цілочислового ділення (/) та взяття залишку від ділення (%):

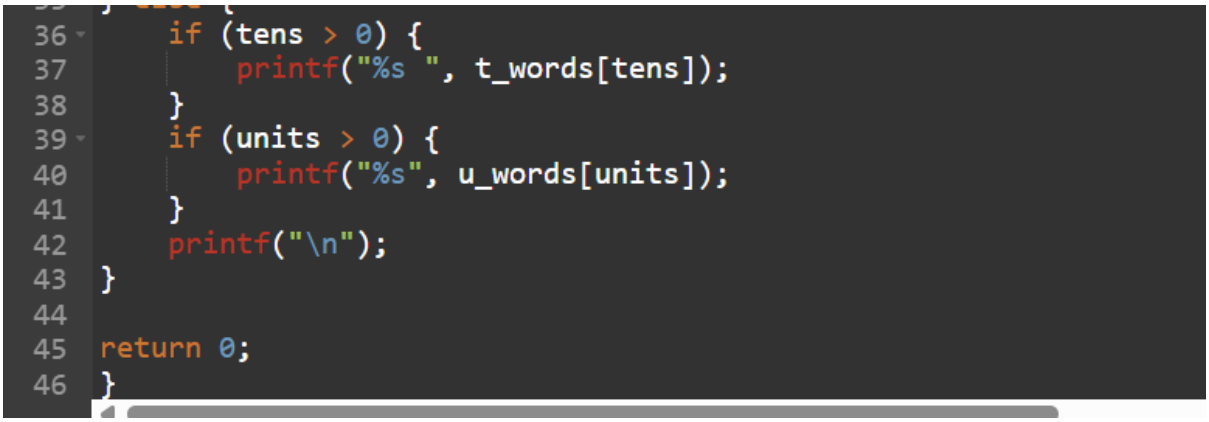
Сотні: hundreds = num / 100 (отримання першої цифри);
Десятки: tens = (num / 10) % 10 (отримання середньої цифри);
Одиниці: units = num % 10 (отримання останньої цифри).

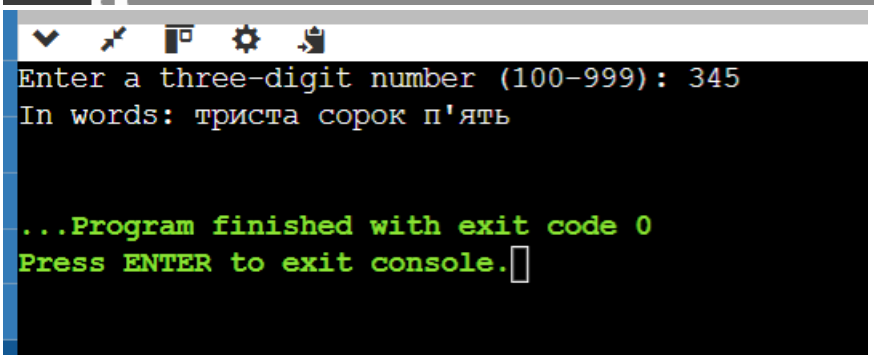
Для текстового формування числа я використала масиви на рядки const char *, які містять текстові еквіваленти для сотень, десятків, одиниць, а також окремий масив **teens для чисел у діапазоні від 10 до 19.**

Логіка побудови підсумкового тексту працює наступним чином: спочатку програма друкує відповідне слово для сотень із масиву h_words. Далі аналізується розряд десятків: якщо tens == 1, це означає, що числа від 10 до 19 формуються одним спеціальним словом (наприклад, «п'ятнадцять»), яке беруть із масиву teens. Якщо ж значення десятків інше, програма окремо виводить назву десятків із масиву t_words (якщо tens > 0) та назву одиниць із масиву u_words (якщо units > 0).

Повний текст розробленого коду має такий вигляд:







https://onlinegdb.com/YiYqWwIm_

<script src="//onlinegdb.com/embed/js/YiYqWwIm_?theme=dark"></script>

Я успішно скомпільовала та виконала дану програму. Під час тестування на числовому значенні 345 програма коректно

сформувала та вивела стрічку «триста сорок п'ять». Помилок під час виконання не виявлено, результат зафіксовано на скріншоті, а код збережено за відповідним онлайн-посиланням.

ВИСНОВОК:

Під час виконання цієї роботи я на практиці розібралася, як працюють розгалуження та умови в мові С. Я навчилася писати алгоритми, які перевіряють правила трикутників і класифікують їх за сторонами та кутами. Крім того, я зрозуміла, як за допомогою звичайного ділення й залишку розбивати числа на розряди (сотні, десятки, одиниці) та використовувати масиви рядків, щоб програма виводила числа прописом. Усі написані програми я самостійно протестувала у середовищі Dev-C++ та OnlineGDB, переконавшись у їхній правильній роботі.